Rob Bickel Agency · Managed Software Services

The Orphaned Software Playbook

Seven steps to regain control of the applications your business depends on, but nobody owns anymore.

For COOs, CFOs, and operations leaders

THE PROBLEM

Orphaned software is software your business runs on, with no one accountable for it.

The builder is gone

A contractor or employee wrote it, then moved on.

Nobody will touch it

Changes feel risky, so requests wait for months.

No one is watching

Outages surface when a customer complains.

Keys are scattered

Logins and API keys live in inboxes and laptops.

It's still billing

Hosting and licenses renew whether it's used or not.

Audits get awkward

Security questionnaires stall on "we're not sure."

01

Find everything

You can't manage what you can't see. Start with a list, not a fix.

- ✓ Pull every software line item from the last 12 months of card and AP statements
- ✓ Ask each department head: "What would stop you working if it went down?"
- ✓ Include scripts, spreadsheets with macros, and integrations, not just apps
- ✓ Record for each: what it does, who uses it, what it costs

Quick win: the finance statement review alone usually surfaces tools nobody knew were still running.

02

Name an owner for each system

"IT" or "the vendor" isn't an owner. A named person is accountable for each system's health.

- ✓ Assign a business owner (who decides) and a technical owner (who maintains)
- ✓ Flag every system where the technical owner is "nobody" or has left
- ✓ Rank those by business impact: revenue, customers, compliance

Rule of thumb: if one person leaving would leave a system with no one able to change it, that's your first priority.

03

Secure the keys

Access is the first thing lost when someone leaves, and the hardest to recover.

- ✓ Move domains, hosting, code repositories, and vendor accounts into company-owned logins
- ✓ Store credentials in a shared password manager, not inboxes or laptops
- ✓ Rotate any API key or password a departed person had access to
- ✓ Scan code repositories for secrets committed by accident

04

Turn the lights on

Learn about problems before your customers do.

- [ ] Add uptime monitoring with alerts that reach a real person
- [ ] Capture application errors in one place
- [ ] Confirm backups exist, then test a restore. An untested backup is a guess.
- [ ] Write down what "down" costs per hour, so you know how fast to respond

05

Write it down

Documentation turns personal knowledge into company property.

- ✓ A one-page summary per system: purpose, users, dependencies, costs
- ✓ Runbooks for the common tasks: deploy, restart, restore, rotate a key
- ✓ A simple diagram of how systems pass data to each other
- ✓ Keep it next to the code, and update it when things change

Test: could a competent engineer who has never seen this system deploy a fix using only what's written down?

06

Cut what you don't need

Every system you retire is one less thing to secure, patch, and pay for.

- **Keep:** it works and the business depends on it
- **Consolidate:** several tools doing one job
- **Rebuild:** it's critical but too fragile to maintain
- **Retire:** nobody will miss it. Export the data first.

07

Put it under management

The first six steps restore control. Ongoing ownership keeps it.

- ✓ Patch dependencies and runtimes on a schedule, not after an incident
- ✓ Handle change requests through one accountable owner
- ✓ Review the portfolio quarterly: cost, risk, and what to cut next
- ✓ Decide who owns this long-term: an internal hire or a managed service

Want someone to own this for you?

We provide managed services for the software your business runs on: monitored, patched, documented, and improved. I lead every engagement personally, and our onshore and nearshore engineers support you, all under one enterprise service agreement with flexible scopes of work and monthly invoicing.

bickel.rob@gmail.com

robbickel.com